

update-markdown-uml Implementation Plan

PHASE 1 — Refactor: Re-home link validation out of tooling-core

Step 1 — Migrate `update-markdown-toc` from Jest to Vitest

- Add `vitest.config.ts` (same pattern as `nx-graph-to-mermaid`)
- Add `tsconfig.test.json`
- Add `vitest` to `devDependencies`, remove `jest`
- Delete `jest.config.js`
- Add `tests/smoke.test.ts` — dummy test to verify wiring
- Update `project.json` test target:

```
cd update-markdown-toc && npx vitest run --config vitest.config.ts && bash scripts/run-all-tests.sh
```

Step 2 — Write failing tests

Two test files, both fail until implementation is complete:

tooling-core/tests/cli/runCli.test.ts — new tests:

- `afterRun` is called when defined on descriptor
- `afterRun` is not called when not defined on descriptor
- `afterRun` receives the files that were processed
- `afterRun` is not called when `--help` exits early

update-markdown-toc/tests/cli/descriptorAfterRun.test.ts — new file:

- `afterRun` is defined on the toc descriptor
- `afterRun` calls `runLinkValidation` in check mode
- `afterRun` does not call `runLinkValidation` in update mode
- `afterRun` passes `validateExternalLinksLocal` mapped to `validateExternalLinks` in the call to `runLinkValidation`
- `afterRun` passes `linkTimeoutMsLocal` mapped to `linkTimeoutMs` in the call to `runLinkValidation`
- `validateExternalLinks` is forced to `false` in the standard config returned by `parseOptions` so `tooling-core`'s built-in call is a no-op
- `runLinkValidation` is called exactly once per check-mode run — not twice

Step 3 — Add suffixed fields to `TocRunConfig`

Create update-markdown-toc/src/cli/TocRunConfig.ts:

```
export interface TocRunConfig extends RunConfig {  
  validateExternalLinksLocal: boolean;
```

```
    linkTimeoutMsLocal: number;
  }
```

Wire `update-markdown-toc` to use these fields internally — in `descriptor.ts` `parseOptions`, consuming `--no-external-link-check/-n` and `--link-timeout-ms/-l` into the `Local` fields. Force `validateExternalLinks: false` on the standard config to disable `tooling-core`'s built-in call. `tooling-core` fields still present and unchanged. Both sets of fields coexist — no shadowing since they have distinct names. Tests from Step 2 now pass.

Step 4 — Verify

Run full test suite. Confirm Step 2 tests pass. No regressions. `tooling-core` fields still present — this step is green-light only, no code changes.

Step 5 — Remove from `tooling-core`, add `afterRun` hook

`tooling-core/src/cli/types.ts` — add to `PluginDescriptor`:

```
  afterRun?(this: void, files: string[], config: TConfig): Promise<void>;
```

`tooling-core/src/cli/runCli.ts` — remove `runLinkValidation` call, add:

```
  if (options.descriptor.afterRun !== undefined) {
    await options.descriptor.afterRun(targets.files, config);
  }
```

`tooling-core/src/repository/types.ts` — remove `validateExternalLinks` and `linkTimeoutMs` from `RunConfig`

`tooling-core/src/cli/parseStandardCli.ts` — remove defaults and parsing for both fields

`tooling-core/src/cli/generateHelp.ts` — remove `-n/--no-external-link-check` and `-l/--link-timeout-ms` from `STANDARD_OPTIONS`

`update-markdown-toc/src/cli/descriptor.ts` — wire `afterRun`:

```
  async afterRun(files, config) {
    if (config.runMode !== "check") return;
    await runLinkValidation(files, {
      ...config,
      validateExternalLinks: config.validateExternalLinksLocal,
      linkTimeoutMs: config.linkTimeoutMsLocal,
    });
  }
```

Fix all `RunConfig` construction sites in `tooling-core/tests/` that currently set

`validateExternalLinks` and `linkTimeoutMs`.

Step 6 — Rename suffixed fields to final names

In `update-markdown-toc/src/cli/TocRunConfig.ts`:

- ``validateExternalLinksLocal`` → ``validateExternalLinks``
- ``linkTimeoutMsLocal`` → ``linkTimeoutMs``

Update all references in `descriptor.ts`, `descriptorAfterRun.test.ts`, and any other files in `update-markdown-toc` that reference the suffixed names.

Step 7 — Remove workaround from ``nx-graph-to-mermaid``

In `nx-graph-to-mermaid`'s `parseOptions` — remove the `validateExternalLinks: false` override. No longer needed since `validateExternalLinks` no longer exists on `RunConfig`.

PHASE 2 — Implement ``update-markdown-uml``

Step 8 — Source discovery (``src/discovery/discoverLeafPackages.ts``)

Pure function:

```
function discoverLeafPackages(  
  sourceRoot: string,  
  skipTestPatterns: string[]  
): string[]
```

- Resolve source root: use ``src/`` if present, else package root, else ``--source`` override
- Enumerate subdirectories under source root
- For each directory: collect ``.ts`` files, filter out any matching patterns in

`skipTestPatterns` using `micromatch`

- If zero ``.ts`` files remain after filtering → not a leaf, drop from result
- Returns sorted list of absolute leaf directory paths

Default `skipTestPatterns`: `["*.test.ts", "*.spec.ts"]`

`UmlRunConfig` additions:

```
skipTestPatterns: string[];
```

`descriptor.ts` additions:

```
{  
  flag: "--skip-test-patterns",  
  description: "Glob patterns for test files to exclude from leaf package detection (default: *.test.ts, *.spec
```

```

    requiresValue: true,
    valueName: "pattern1,pattern2,...",
  },
  {
    flag: "-s",
    description: "Short form of --skip-test-patterns",
    requiresValue: true,
    valueName: "pattern1,pattern2,...",
  },

```

update-markdown-uml/package.json additions:

```

"dependencies": {
  "micromatch": "^4.0.0"
},
"devDependencies": {
  "@types/micromatch": "^4.0.0"
}

```

Unit-testable with temp dirs.

Step 9 — Package description reading (`src/discovery/readPackageDescription.ts`)

Pure function:

```
function readPackageDescription(leafDir: string): string | undefined
```

- Looks for `\_PACKAGE\_INFO.md` in `leafDir`
- If absent → returns `undefined`
- If present → returns first non-empty line, or `undefined` if file is blank
- Remainder of file ignored

Trivially unit-testable.

Step 10 — Inter-package dependency analysis (`src/analysis/analyzeImportDependencies.ts`)

Uses ts-morph to resolve cross-package imports. Full transitive closure — if

cli → repository → util, all three edges are returned.

```

function analyzeImportDependencies(
  leafDirs: string[],
  sourceRoot: string
): Array<{ from: string; to: string }>

```

- `from` and `to` are leaf directory names (not full paths)
- Returns deduplicated, sorted edges
- All edges represent `..>` call/import dependencies with `: calls` label in the

overview diagram

Unit-testable with fixture source trees.

Step 11 — Flowchart diagram generation (`src/generators/buildPackagesFlowchart.ts`)

Pure function:

```
function buildPackagesFlowchart(  
  packages: string[],  
  edges: Array<{ from: string; to: string }>  
) : string
```

- Produces `flowchart TB`
- One `subgraph` per leaf package containing its type names
- Inter-package arrows use `-->` (Mermaid flowchart syntax, not classDiagram syntax)
- No I/O, fully unit-testable

Step 12 — Packages table generation (`src/generators/buildPackagesTable.ts`)

Pure function:

```
function buildPackagesTable(  
  packages: string[],  
  descriptions: Map<string, string | undefined>  
) : string
```

- Produces Markdown table with two columns: Package, Description
- Package names are clickable links to corresponding package details section anchors
- Missing description renders as `TBD` — visible signal that description is absent
- Blank first line in `\_PACKAGE\_INFO.md` suppresses `TBD` (renders empty)

Unit-testable.

Step 13 — Per-package class diagram generation (`src/generators/buildPackageClassDiagram.ts`)

Uses ts-morph exclusively (tsuml2 kept in package.json for now but not used).

Extracts types, structural relationships, and call dependencies per leaf dir.

Produces classDiagram blocks with typed edge data mapped to three distinct arrow styles:

- structural `extends` → `--|>`
- structural `implements` → `..|>`
- call/usage dependencies → `..>` with `: calls` label

Integration-testable against a fixture package.

Step 14 — Multi-marker injection (`src/markdown/injectUmlSections.ts`)

Local implementation of multi-marker inject supporting three marker pairs:

```
<!-- UML:packages:START -->      / <!-- UML:packages:END -->
<!-- UML:packages-table:START --> / <!-- UML:packages-table:END -->
<!-- UML:package-details:START --> / <!-- UML:package-details:END -->
```

Per the design doc, `findMarker/injectSections` should eventually migrate to `tooling-core` — implement locally first, extract later. Each marker pair injected independently. Check mode compares each region independently — stale reporting identifies which specific region drifted.

Step 15 — Wire `UmlFileProcessor`

Compose Steps 8–14 into a real `FileProcessor`, replacing the current no-op stub in `bin/update-markdown-uml.js`. First end-to-end runnable state. Also:

- Confirm `--recursive` is explicitly rejected in `validateUmlConfig` with a clear error message

Step 16 — Check mode

Per-region byte-for-byte comparison. Each of the three marker regions compared independently. Stale reporting identifies which specific region drifted. Any stale region causes non-zero exit.

Step 17 — `autogen-markdown-doc` integration

Replace the current `execSync` stub invocation in `autogen-markdown-doc/bin/autogen-markdown-doc.js` with a proper call to `update-markdown-uml`, passing the markdown file path. Update `autogen-markdown-doc` README to document UML generation as part of the uber-plugin's behavior.